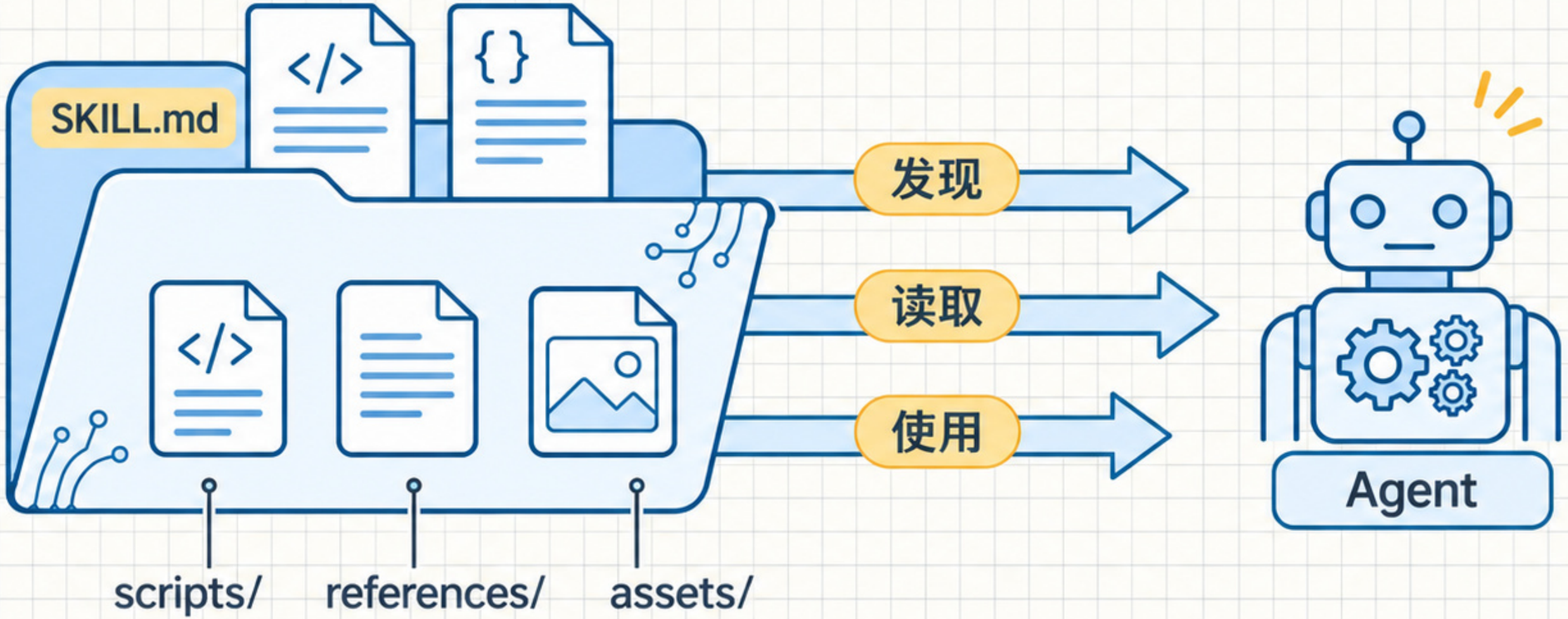
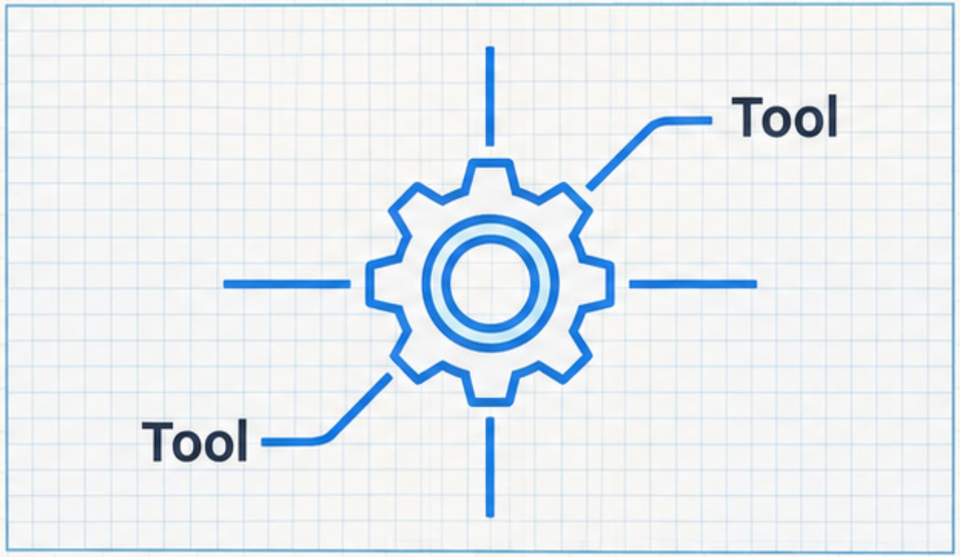
Deep Agents 实战

第 7 章 · 上篇：Skills 从概念到上手



Skills 是什么？

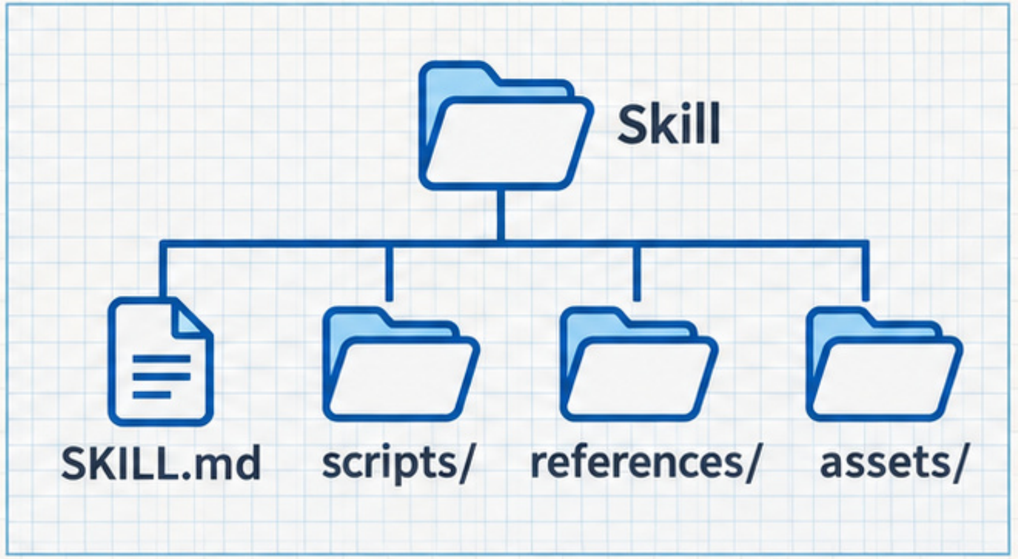
Tool：执行一个具体操作



- 执行一个具体操作

VS

Skill：用说明与资源组织一套任务流程



- 必需：SKILL.md
- 可选：scripts/、references/、assets/

采用 Agent Skills 格式；跨工具复用仍需检查运行环境与工具能力。

SKILL.md 的结构

YAML Frontmatter

必填：name、description

name	1–64 字符，与目录名一致
description	1–1024 字符，说明用途与触发条件

可选：license、compatibility、metadata、allowed-tools

- license
- compatibility
- metadata
- allowed-tools

Markdown 正文

步骤、输入输出、异常处理

```
## 步骤
1. …
2. …
3. …

## 输入输出
• 输入：…
• 输出：…

## 异常处理
• …
```

步骤

完成该技能的执行流程。

输入输出

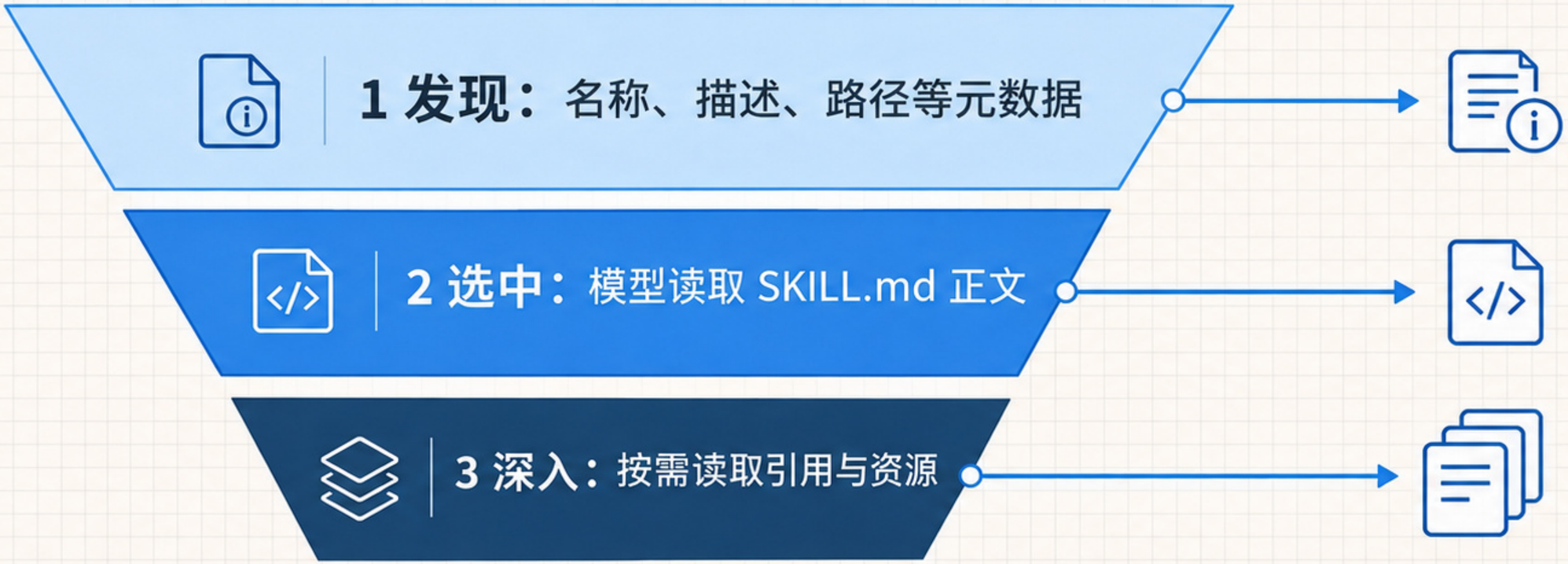
明确输入内容与输出结果。

异常处理

说明可能的异常及应对方式。

⚠ **allowed-tools 是实验性推荐元数据，不是权限沙箱。**

渐进式披露：按需进入上下文



- 扫描时仍需读取文件解析元数据；**不是**只读取 YAML 字节。
- 元数据仍占上下文；正文建议精简，**不是固定 5000-token 硬上限。**

写好 description 的关键

两张卡片的对比示例：

明确：从 PDF 提取文本与表格、填写表单、合并文件；处理 PDF 时使用。

做什么　何时用　关键词

VS

模糊：帮助处理 PDF。



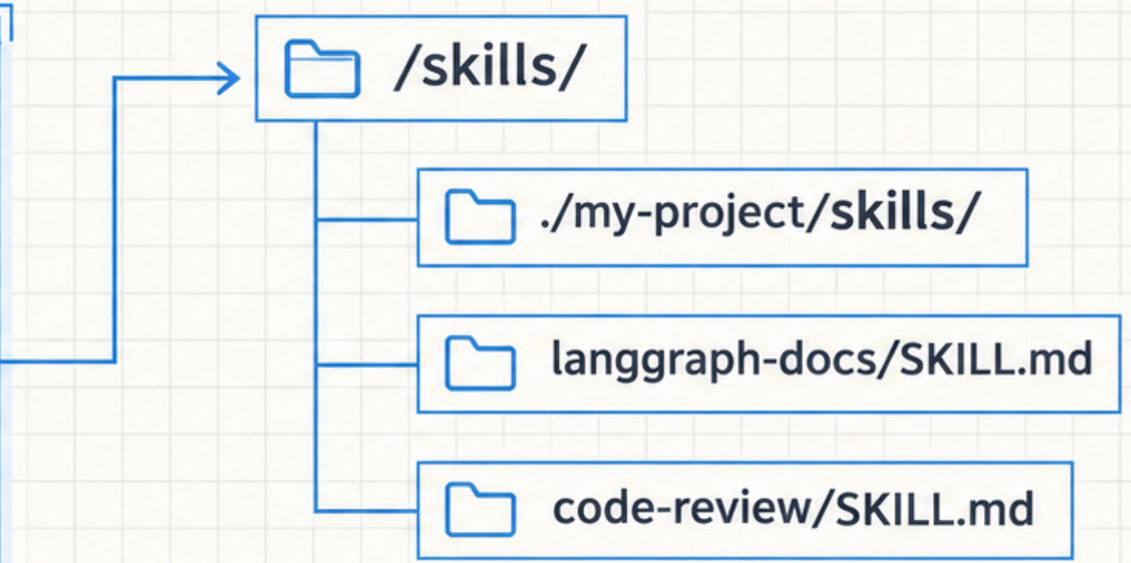
description 是重要匹配信号；模型也能看到名称与路径，并非唯一信息。

在 Deep Agents 中使用 Skills

```python
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

backend = FilesystemBackend(
  root_dir="./my-project", virtual_mode=True
)

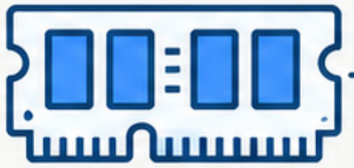
agent = create_deep_agent(
  model=model, backend=backend, skills=["/skills/"]
)
```



- model 需**预先配置**；
- 扫描父目录下已有技能，**不自动创建文件**。

三种后端，三种存储语义

StateBackend



技能文件放入 `state`；同线程恢复取决于 `Checkpointer`。

FilesystemBackend



从配置的 `root_dir` 读取磁盘文件。

StoreBackend



namespace

store

需要 `store` 与明确 `namespace`；共享范围由 `namespace` 决定。

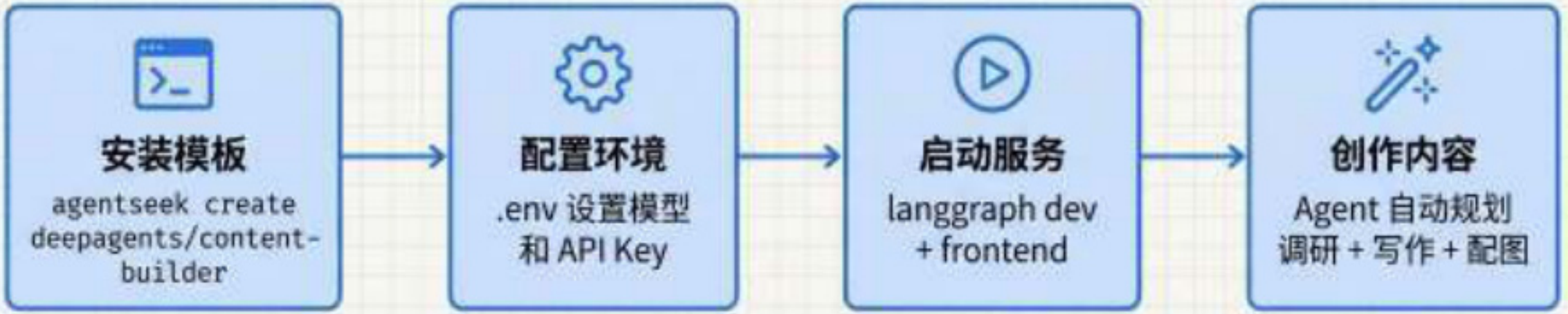
agent

`InMemoryStore` 只在进程生命周期内保存，不是持久数据库。

实战：content-builder 模板

AgentSeek 模板应用 — 内容创作 Agent

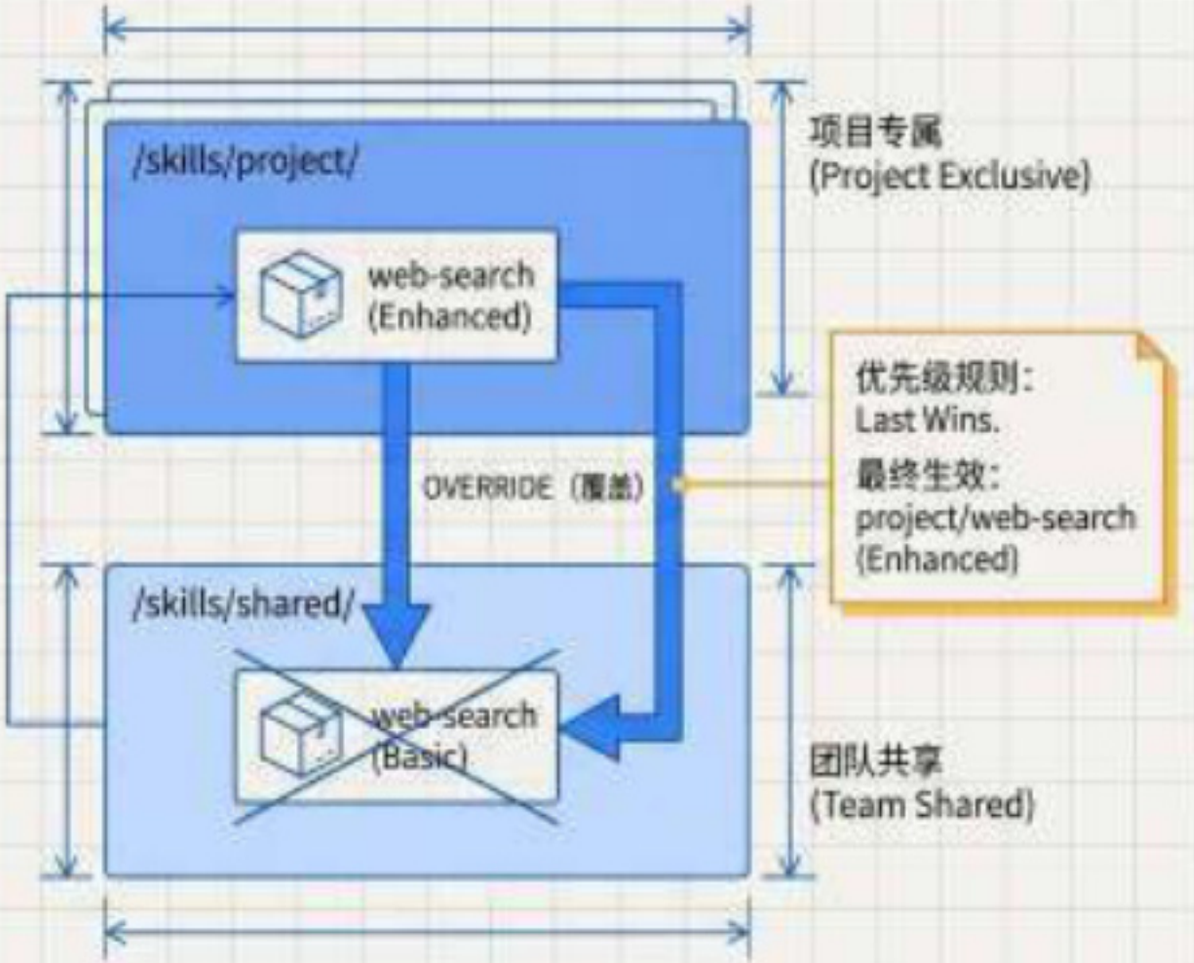
- 内置 Skills：blog-post（博客写作流程）+ social-media（社交媒体内容）
- 品牌记忆：AGENTS.md 定义品牌声音与写作风格
- 子 Agent：researcher（Tavily 搜索）协助素材调研
- 图片生成：Gemini 生成封面和社交配图
- 安装：agentseek create deepagents/content-builder



多源 Skills 与优先级

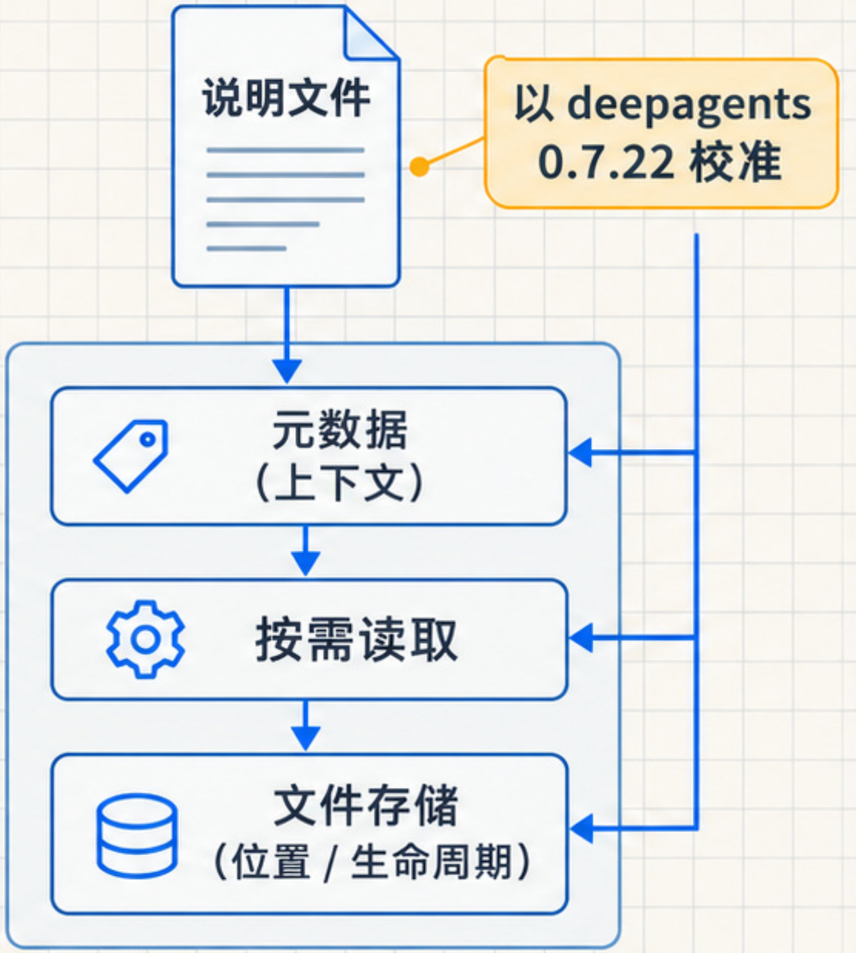
后面的覆盖前面的（last wins）

- skills 参数接受多个路径：["/skills/shared/", "/skills/project/"]
- 同名 Skill 出现在多个目录时，后面的优先
- 分层管理：团队共享（基础）+ 项目专属（覆盖）
- 示例：两个目录都有 "web-search" Skill，project/ 下的生效



本篇回顾

- **Skill：** 说明文件 + 可选资源
- **发现：** 元数据先进入上下文
- **使用：** 按需读取，不等于自动执行
- **存储：** 后端决定文件位置与生命周期



下一篇： **脚本执行、子 Agent 与权限边界**